

</>

The Anuj DECODED

</>

```
01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
};
```



```
01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver
```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

115 · Distinct Subsequences

LeetCode · Classic Hard DP

Hard

String

Dynamic Programming

1. What Type of Question Is This?

This is a classic **two-string dynamic programming** problem, in the same family as Longest Common Subsequence and Edit Distance. Whenever a problem asks you to "count the number of ways" to match, delete, or transform one string into another character by character, the standard move is a **2-D DP table indexed by a prefix of each string**: $dp[i][j]$ = some count involving the first i characters of one string and the first j characters of the other. Here specifically: $dp[i][j]$ will represent **how many distinct subsequences of the first i characters of s equal the first j characters of t** .

2. What Is Given?

- Two strings s and t .
- A **subsequence** of s is what remains after deleting zero or more characters from s without changing the order of the remaining characters (e.g. "rbt" is a subsequence of "rabbit").
- Two subsequences are considered **distinct** if they were formed by deleting a different *set of positions* from s — even if the resulting characters spell the exact same string. That's why "rabbbbit" can form "rabbit" in 3 different ways: there are 3 b's to choose 2 from, each choice deletes a different position even though every resulting string reads "rabbit".

3. What Do We Need to Find?

Return the **number of distinct subsequences of s that equal t exactly**. The problem guarantees the answer fits in a 32-bit signed integer, so no special big-number handling is required.

Function Signature

■ C++

```
class Solution {
public:
    int numDistinct(string s, string t) {

    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input: s = "rabbbit", t = "rabbit"
Output: 3
```

s = "rabbbit" has three b's in a row (positions 2, 3, 4) but t = "rabbit" only needs two b's. Any 2 of those 3 b's can be kept while the third is deleted, and the rest of the letters (r, a, i, t) are used as-is — giving exactly **3** distinct ways to delete down to "rabbit": keep b's at positions (2,3), (2,4), or (3,4), deleting the third one each time.

Example 2

```
Input: s = "babgbag", t = "bag"
Output: 5
```

s = "babgbag" contains the letters needed for t = "bag" (a 'b', then an 'a', then a 'g') scattered across several possible positions, and there turn out to be **5** distinct ways to pick a b, then a later a, then a later g, that together spell "bag".

Constraints

- $1 \leq s.length, t.length \leq 1000$
- s and t consist of English letters.
- The test cases are generated so that the answer fits in a 32-bit signed integer.

Both strings can be up to 1000 characters, so a solution needs to be roughly $O(|s| \times |t|)$ — about 10^6 operations in the worst case — to run comfortably in time.

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Define the state around two prefixes.** Let $dp[i][j]$ = the number of distinct subsequences of $s[0..i-1]$ (the first i characters of s) that equal $t[0..j-1]$ (the first j characters of t). The final answer is $dp[n][m]$, where $n = |s|$ and $m = |t|$.
- **Step 2 — Think about the last character of s 's prefix, $s[i-1]$.** When building a subsequence of $s[0..i-1]$ that matches $t[0..j-1]$, character $s[i-1]$ is either used as the final matched character, or it isn't used at all (skipped/deleted). Those are the only two possibilities, and they never overlap, so the total count is their sum.
- **Step 3 — "Skip $s[i-1]$ " always contributes $dp[i-1][j]$.** If $s[i-1]$ isn't part of the subsequence, then everything must come from $s[0..i-2]$ matching $t[0..j-1]$ — exactly $dp[i-1][j]$, regardless of what $s[i-1]$ or $t[j-1]$ actually are.
- **Step 4 — "Use $s[i-1]$ " only makes sense if $s[i-1] == t[j-1]$.** If the characters match, using $s[i-1]$ as the match for $t[j-1]$ leaves the rest of the job as matching $s[0..i-2]$ against $t[0..j-2]$ — exactly $dp[i-1][j-1]$. If they don't match, this option contributes 0 (you simply can't use $s[i-1]$ to complete $t[j-1]$).
- **Step 5 — Combine into one recurrence.** $dp[i][j] = dp[i-1][j] + (s[i-1] == t[j-1] ? dp[i-1][j-1] : 0)$.
- **Step 6 — Nail the base cases.** $dp[i][0] = 1$ for every i (there is exactly one way to match the empty string t : delete everything). $dp[0][j] = 0$ for every $j > 0$ (you can't match a non-empty t out of an empty s).

Which Algorithm Is Best?

The recurrence in Step 5 only ever looks at **row $i-1$** when computing row i — it never needs any earlier row than that. That means the full 2-D table is more memory than necessary: a single 1-D array reused across rows, updated carefully from right to left, gives the exact same answer using $O(m)$ space instead of $O(n \cdot m)$. This is a very common DP optimization (the same trick used in 0/1 knapsack), and it's the natural "optimal" endpoint after first getting the recurrence right with plain recursion and then with a full memo table.

Approach 1 — Brute Force (Plain Recursion, Try Both Choices at Every Position)

Idea: Directly translate Step 2 of the workflow into recursive code: at every position i in s , either skip $s[i]$ entirely, or (if it matches the character t needs next) use it, and recurse in both directions, summing the number of ways. No results are cached, so the same (i, j) pair can be recomputed many times from different call paths.

Steps

- Define $\text{countWays}(i, j)$: the number of ways to match $t[j:]$ using characters from $s[i:]$.
- Base case: if j reaches the end of t , one full match has been completed — return 1.
- Base case: if i reaches the end of s (but j hasn't finished), there's nothing left to match with — return 0.
- Recursive case: always add the "skip $s[i]$ " branch, $\text{countWays}(i+1, j)$.
- If $s[i] == t[j]$, also add the "use $s[i]$ " branch, $\text{countWays}(i+1, j+1)$.
- Return the sum of whichever branches applied; the top-level call is $\text{countWays}(0, 0)$.

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // countWays(i, j) = number of distinct subsequences of s[i..] matching t[j..]
    int countWays(string& s, string& t, int i, int j) {
        if (j == (int)t.size()) return 1; // matched all of t — one valid way
        if (i == (int)s.size()) return 0; // ran out of s, t not fully matched

        int skip = countWays(s, t, i + 1, j); // don't use s[i]
        int take = 0;
        if (s[i] == t[j]) take = countWays(s, t, i + 1, j + 1); // use s[i] to match t[j]

        return skip + take;
    }

    int numDistinct(string s, string t) {
        return countWays(s, t, 0, 0);
    }
};

int main() {
    string s = "rabbbit", t = "rabbit";
    Solution sol;
    cout << sol.numDistinct(s, t) << endl; // Expected output: 3
    return 0;
}
```

■ Python

```
import sys

class Solution:
    def numDistinct(self, s, t):
        sys.setrecursionlimit(10000)
        return self.countWays(s, t, 0, 0)

    # countWays(i, j) = number of distinct subsequences of s[i:] matching t[j:]
    def countWays(self, s, t, i, j):
        if j == len(t):
```

```

        return 1                # matched all of t – one valid way
    if i == len(s):
        return 0                # ran out of s, t not fully matched

    skip = self.countWays(s, t, i + 1, j)    # don't use s[i]
    take = 0
    if s[i] == t[j]:
        take = self.countWays(s, t, i + 1, j + 1) # use s[i] to match t[j]

    return skip + take

if __name__ == "__main__":
    s, t = "rabbbit", "rabbit"
    print(Solution().numDistinct(s, t))    # Expected output: 3

```

■ JavaScript

```

class Solution {
    numDistinct(s, t) {
        return this.countWays(s, t, 0, 0);
    }

    // countWays(i, j) = number of distinct subsequences of s[i:] matching t[j:]
    countWays(s, t, i, j) {
        if (j === t.length) return 1;        // matched all of t – one valid way
        if (i === s.length) return 0;        // ran out of s, t not fully matched

        const skip = this.countWays(s, t, i + 1, j);    // don't use s[i]
        let take = 0;
        if (s[i] === t[j]) take = this.countWays(s, t, i + 1, j + 1); // use s[i] to match t[j]

        return skip + take;
    }
}

const s = "rabbbit", t = "rabbit";
console.log(new Solution().numDistinct(s, t));    // Expected output: 3

```

Complexity: Each call branches into up to 2 further calls, and the recursion can go $|s|$ levels deep, so the number of calls grows like $O(2^n)$ in the worst case (e.g. when many characters of s match the needed character of t). With $|s|$ up to 1000, this is astronomically slow and will time out — it's only practical for tiny inputs, but it is the clearest possible statement of the recurrence.

Approach 2 — Better (Top-Down DP with Memoization)

Idea: The brute-force recursion above only ever depends on the pair (i, j) — the exact same subproblem gets solved repeatedly along different branches. Cache each (i, j) result the first time it's computed, and every later call for the same pair becomes an $O(1)$ lookup instead of a fresh recursive exploration.

Steps

- Keep the exact same recursive structure and base cases as Approach 1.
- Add a 2-D memo table (or a cache decorator) indexed by (i, j) .
- Before doing any recursive work, check whether `memo[i][j]` already has an answer; if so, return it immediately.
- After computing skip + take, store the result in `memo[i][j]` before returning it.
- The top-level call is still `countWays(0, 0)`.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<vector<int>> memo;

    int countWays(string& s, string& t, int i, int j) {
        if (j == (int)t.size()) return 1;
        if (i == (int)s.size()) return 0;
        if (memo[i][j] != -1) return memo[i][j]; // already computed this state

        int skip = countWays(s, t, i + 1, j);
        int take = 0;
        if (s[i] == t[j]) take = countWays(s, t, i + 1, j + 1);

        return memo[i][j] = skip + take;
    }

    int numDistinct(string s, string t) {
        int n = s.size(), m = t.size();
        memo.assign(n + 1, vector<int>(m + 1, -1));
        return countWays(s, t, 0, 0);
    }
};

int main() {
    string s = "rabbbit", t = "rabbit";
    Solution sol;
    cout << sol.numDistinct(s, t) << endl; // Expected output: 3
    return 0;
}
```

■ Python

```
from functools import lru_cache
import sys

class Solution:
    def numDistinct(self, s, t):
        sys.setrecursionlimit(10000)

        @lru_cache(maxsize=None)
        def countWays(i, j):
```

```

        if j == len(t):
            return 1
        if i == len(s):
            return 0

        skip = countWays(i + 1, j)
        take = 0
        if s[i] == t[j]:
            take = countWays(i + 1, j + 1)

        return skip + take

    result = countWays(0, 0)
    countWays.cache_clear()
    return result

if __name__ == "__main__":
    s, t = "rabbbit", "rabbit"
    print(Solution().numDistinct(s, t))    # Expected output: 3

```

■ JavaScript

```

class Solution {
    numDistinct(s, t) {
        const n = s.length, m = t.length;
        const memo = Array.from({ length: n + 1 }, () => new Array(m + 1).fill(-1));

        const countWays = (i, j) => {
            if (j === m) return 1;
            if (i === n) return 0;
            if (memo[i][j] !== -1) return memo[i][j];    // already computed this state

            const skip = countWays(i + 1, j);
            let take = 0;
            if (s[i] === t[j]) take = countWays(i + 1, j + 1);

            return (memo[i][j] = skip + take);
        };

        return countWays(0, 0);
    }
}

const s = "rabbbit", t = "rabbit";
console.log(new Solution().numDistinct(s, t));    // Expected output: 3

```

Complexity: There are only $(n+1) \times (m+1)$ distinct (i, j) pairs, and each is computed exactly once thanks to the cache, with $O(1)$ work per pair beyond the two recursive lookups — $O(n \cdot m)$ time overall. Space is also $O(n \cdot m)$ for the memo table (plus $O(n)$ recursion stack depth), comfortably handling the 1000×1000 worst case (about 10^6 table entries).

Approach 3 — Optimal (Bottom-Up DP Compressed to a 1-D Array)

Idea: The recurrence $dp[i][j] = dp[i-1][j] + (\text{match} ? dp[i-1][j-1] : 0)$ only ever reads from row $i-1$, never anything earlier. So instead of storing every row, keep a single array $dp[0..m]$ that represents "the row so far", and update it in place for each new i — as long as you walk j from high to low, $dp[j-1]$ hasn't been overwritten yet when you need its old (row $i-1$) value.

Steps

- Allocate a single 1-D array dp of size $m+1$, with $dp[0] = 1$ (empty t , one way) and everything else 0 (matches the $i=0$ base row).
- For each character of s (i from 1 to n): walk j from m down to 1.
- If $s[i-1] == t[j-1]$: $dp[j] += dp[j-1]$ — this folds in the "use $s[i-1]$ " option, since $dp[j-1]$ at this point still holds the previous row's value (we haven't reached index $j-1$ yet in this downward sweep).
- If they don't match, $dp[j]$ is left untouched — it already equals $dp[i-1][j]$, which is exactly the "skip $s[i-1]$ " contribution.
- After processing all of s , $dp[m]$ holds the final answer.

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int numDistinct(string s, string t) {
        int n = s.size(), m = t.size();

        vector<int> dp(m + 1, 0);
        dp[0] = 1; // matching empty t: always 1 way

        for (int i = 1; i <= n; i++) {
            // walk j from high to low so dp[j-1] still holds the PREVIOUS row's value
            for (int j = m; j >= 1; j--) {
                if (s[i - 1] == t[j - 1]) {
                    dp[j] += dp[j - 1];
                }
                // if characters don't match, dp[j] already equals dp[i-1][j] — no change
            }
        }
        return dp[m];
    }
};

int main() {
    string s = "rabbbit", t = "rabbit";
    Solution sol;
    cout << sol.numDistinct(s, t) << endl; // Expected output: 3
    return 0;
}
```

■ Python

```
class Solution:
    def numDistinct(self, s, t):
        n, m = len(s), len(t)

        dp = [0] * (m + 1)
```

```

dp[0] = 1                                # matching empty t: always 1 way

for i in range(1, n + 1):
    # walk j from high to low so dp[j-1] still holds the PREVIOUS row's value
    for j in range(m, 0, -1):
        if s[i - 1] == t[j - 1]:
            dp[j] += dp[j - 1]
        # if characters don't match, dp[j] already equals dp[i-1][j] – no change

return dp[m]

if __name__ == "__main__":
    s, t = "rabbbit", "rabbit"
    print(Solution().numDistinct(s, t))    # Expected output: 3

```

■ JavaScript

```

class Solution {
    numDistinct(s, t) {
        const n = s.length, m = t.length;

        const dp = new Array(m + 1).fill(0);
        dp[0] = 1;                                // matching empty t: always 1 way

        for (let i = 1; i <= n; i++) {
            // walk j from high to low so dp[j-1] still holds the PREVIOUS row's value
            for (let j = m; j >= 1; j--) {
                if (s[i - 1] === t[j - 1]) {
                    dp[j] += dp[j - 1];
                }
                // if characters don't match, dp[j] already equals dp[i-1][j] – no change
            }
        }
        return dp[m];
    }
}

const s = "rabbbit", t = "rabbit";
console.log(new Solution().numDistinct(s, t));    // Expected output: 3

```

Complexity: Still $O(n \cdot m)$ time (every (i, j) pair is still visited once), but only $O(m)$ extra space instead of $O(n \cdot m)$ — a significant reduction in memory for the same 1000×1000 worst case, and the classic "roll the DP table into one array" pattern worth recognizing whenever a recurrence only looks one row back.

Side-by-Side Comparison

1. Brute Force	Plain recursion: skip or use $s[i]$ at every step, no caching	$O(2^n)$ — TLEs bad	only recursion stack
2. Better	Top-down DP: memoize every (i, j) result the first time it's computed	$O(n \cdot m)$	$O(n \cdot m)$ memo table
3. Optimal	Bottom-up DP compressed to a single 1-D array, updated right to left	$O(n \cdot m)$	$O(m)$, no 2-D table

Key Takeaways to Remember

- **"Count the number of ways to match/transform one string into another"** is the signature phrase for a 2-D DP over prefixes of both strings — define $dp[i][j]$ in terms of the first i and first j characters, then figure out the recurrence from the last character of each prefix.
- At the last character of the first string's prefix, there are usually exactly two options — **use it or skip it** — and the DP recurrence is simply the sum of whichever options are valid.
- Distinct subsequences are counted by **position**, not by resulting string content — identical-looking results from different deleted positions all count separately.
- Whenever a DP recurrence for row i only reads row $i-1$ (never anything earlier), you can almost always **compress the table to a single rolling array**, provided you update it in the right direction (here, right to left) so you don't overwrite a value before you need it.
- All three tiers were validated against **Example 1** ($s = \text{"rabbbit"}, t = \text{"rabbit"} \rightarrow 3$) across C++, Python, and JavaScript, and checked against Example 2 ($s = \text{"babgbag"}, t = \text{"bag"} \rightarrow 5$) as well.